UXL OPEN SOURCE WORKING GROUP

Adapting UXL projects to agentic AI

Quarterly meeting

Wednesday, October 28, 2026 · 12:00 p.m. ET

John Melonakos · Intel, UXL OS WG Co-chair

UXL
Unified Acceleration Foundation

Agenda

60 minutes. Topics from the Oct 15 call for agenda items, plus Biagio's topics.

Time (ET)	Topic	Lead	Min
12:00	Welcome, new representatives, opens	John	5
12:05	UXL skills repo: walkthrough with oneDNN as the example	John	15
12:20	Skills roundtable: what each project wants from it	All	12
12:32	oneAPI plugins under UXL, and a SYCL kernel-writing skill	Biagio Cosenza	12
12:44	Open floor: project updates and agentic-AI ideas	Project leads	8
12:52	oneAPI DevSummit content (CFP closes Oct 31)	All	6
12:58	Close	John	2

Welcome

Each project chooses its own representative. There is no formal process.

New this meeting

- **Sergey Yakovlev** joins as the oneDAL representative. Nikolay Petrov stays a oneDAL maintainer and UXL contributor.
- **Biagio Cosenza** (University of Salerno, Language SIG chair) brings two topics at 12:30.
- **Mailing list:** lists.uxlfoundation.org/g/open-source-wg

Project	Contact
oneDNN	Vadim Pirogov
oneDAL	Sergey Yakovlev
oneMath	Nick Dingle (Arm)
oneTBB	Michael Voss
oneDPL	Timmie Smith
oneCCL	Polina Meltsin

The UXL skills repo, through oneDNN

Roundtable



What the repo holds

github.com/uxlfoundation/skills · guidance and evaluations that help AI coding agents use UXL libraries

skills/	**What agents load**	One folder per skill: 6 libraries, plus SYCL build/debug and performance validation.
skill-cards/	**What people read**	One Markdown page per skill: status, scope, limits, evidence.
skills.yaml	**The catalog**	One entry per skill. Links the skill folder to its card and records status and review.
evaluation/	**The proof**	Harbor tasks that test agents with and without the skill, plus the dashboard.
docs/maintainer-review/	**The handoff**	Review packet, briefing deck and outreach per project.

Skill vs. skill card

Same subject, different readers

The skill: for agents

skills/uxl-onednn/ (a folder, 4 files)

- Loaded into the agent's context when its description matches the task
- Procedures: first pass, implementation, triage, gotchas
- An output contract: what the agent must report
- Links to official oneDNN docs and benchdnn

The skill card: for people

skill-cards/uxl-onednn.md (one file)

- Never loaded by agents
- Five required sections: Status, Purpose, Supported Tasks, Limitations, Evidence
- oneDNN today: incubating, maintainer review needed
- Sources rechecked 2026-08-25 against v3.13.1

skills.yaml ties them together: `catalog_path: skills/uxl-onednn · skill_card: skill-cards/uxl-onednn.md · status · owner_repo · maintainer_review · last_source_verification`

Where the oneDNN skill lives: four files, one folder

```
skills/uxl-onednn/
├── SKILL.md
├── references/
│   └── official-sources.md
├── evals/
│   └── evals.json
└── agents/
    └── openai.yaml
```

File	What it does
SKILL.md	The skill itself, 57 lines. Frontmatter name and description (the trigger), then Purpose, First Pass, Implementation, Triage, Gotchas, Output Contract.
official-sources.md	Source ledger: docs, spec, benchdnn, verbose mode, release v3.13.1, incident #5732, and the rule for refreshing it.
evals.json	4 prompt evals with must-include / must-not-include phrases.
openai.yaml	Display name and default prompt for Codex-style agents.

How the oneDNN skill is evaluated today

Two layers. Every grader is a deterministic script; there is no LLM judge.

1. Prompt evals

evals/evals.json: 4 prompts

- Pass if the answer contains required words (e.g. “layout”, “reorder”, “benchdnn”) and avoids banned ones (“guaranteed speedup”)
- CI checks only the file’s structure; scoring needs saved answers

2. Harbor tasks: an agent fixes a real oneDNN problem in a container

Task	Kind	Result
framework-blocked-layout	answer, rubric	**headroom**
convolution-fusion-parity	build + run	ceiling
extra-reorder-regression	build + run	ceiling
matmul-memory-descriptors	build + run	ceiling
benchdnn-no-ref-memory (#5732)	build + run	ceiling
backend-unimplemented-primitive	target device	**planned**

Headroom: the skill raises the score. Ceiling: agents pass with or without it.

What the oneDNN results say so far

Recorded in evaluation/harbor/results/. One model, few attempts: read as direction, not proof.

+0.26

mean score on the blocked-layout task: 0.70 without the skill, 0.96 with it

3 attempts per arm · 2026-08-08-coverage-wave-1.md

−11.6%

tokens per verified success on the benchdnn incident (9 of 9 passed)

3 attempts per arm · 2026-08-13-onednn-benchdnn-calibration.md

+6.5–51%

more tokens with the skill on the three tasks agents already pass

1 attempt per arm · reorder, matmul, conv-fusion

What this means: the skill helps where agents are weak and costs tokens where they are already strong.

The gap: only one oneDNN task discriminates. Harder tasks from real oneDNN incidents are the best way to show value, and maintainers know those incidents.

Reading “Developing”: what should maintainers make of it?

One oneDNN task shows lift; four hit the ceiling. Across all skills: 9 of 41 headroom, 25 ceiling.

Is the skill wrong?

Not on this evidence.

A ceiling means today’s frontier model already solves the task. It says nothing against the guidance.

Ceiling tasks stay as regression checks: they catch an edit that breaks something.

Should we cut parts of it?

Maybe, with evidence. Every line costs tokens each time the skill loads.

Trimming already worked once: the current oneDNN skill uses 68% fewer tokens than the first version on benchdnn, with no loss.

Cut a section when no headroom task drops without it.

Do we need better tasks?

Yes. This is the main lever.

Tasks from real oneDNN incidents that agents get wrong: multi-step, backend-specific, GPU.

Plus negative controls, which check that the skill stays out of unrelated work.

Bottom line: the skill helps on layout reasoning for one strong model. We need more and harder tasks, and more models, before anyone should trust or drop it.

Who takes the test: one agent, one model

Every recorded run used the Codex CLI with gpt-5.6-sol at medium reasoning effort

How a run works

- **Harbor** starts the agent in a container with the task, then a script grades the result.
- **Skill arm:** the skill is installed, and the agent is told “Use the provided uxl-onednn skill.”
- **Pluggable:** the runner takes an agent and a model as parameters. Results record agent, harness, version, model and effort.
- **Small samples:** 1–3 attempts per arm so far.

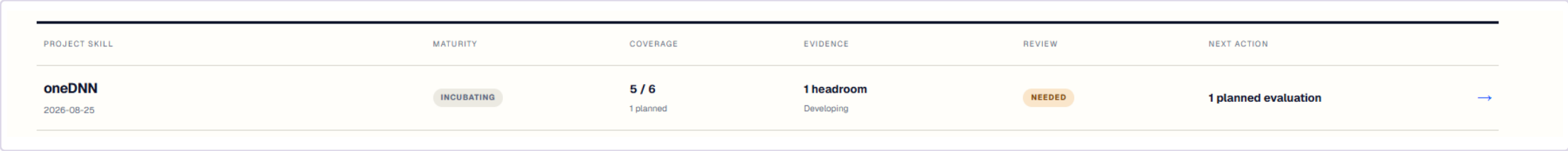
Is it representative? Not yet.

- **One frontier model.** Smaller and local models probably gain more from the skill.
- **One harness.** The repo ships setup for Claude Code, Copilot, Cursor, Gemini CLI and others, but none are tested.
- **No discovery test.** Agents are told to use the skill. Real agents must pick it from its description.
- **Cost:** each arm costs real tokens. Broader coverage needs members to contribute runs.

Question for the room: which agents and models do your users actually run?

Dashboard 1: the portfolio row

uxlfoundation.github.io/skills · one row per skill, rebuilt on every merge that touches skills or tasks

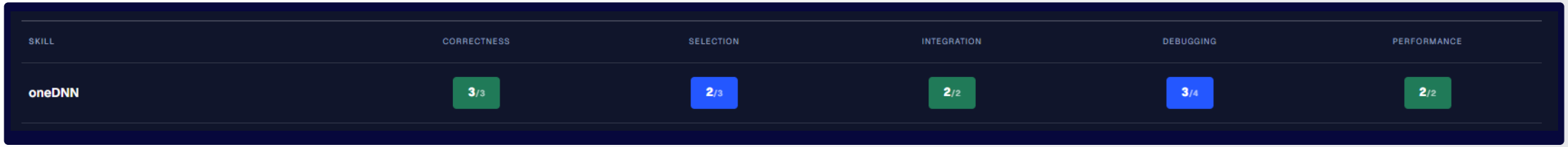


PROJECT SKILL	MATURITY	COVERAGE	EVIDENCE	REVIEW	NEXT ACTION
oneDNN 2026-08-25	INCUBATING	**5 / 6** 1 planned	**1 headroom** Developing	NEEDED	**1 planned evaluation** →

Column	What it means	oneDNN
Project skill	Skill name, and the date its sources were last rechecked against upstream docs	2026-08-25
Maturity	Catalog status: incubating → pilot → reviewed → project-owned	incubating
Coverage	Evaluation tasks implemented / tasks targeted; planned tasks shown below	5 / 6, 1 planned
Evidence	Number of tasks where the skill measurably raises the score (headroom). 2+ = Strong, 1 = Developing, 0 = Needs evidence	1 headroom · Developing
Review	Has the owning project's maintainer reviewed the skill?	needed
Next action	Planned evaluations if any remain; otherwise schedule maintainer review	1 planned evaluation

Dashboard 2: the capability heatmap

Is each of the five kinds of behavior covered by at least one task? It shows coverage, not whether the skill helps.



Column	What the skill should teach (oneDNN)	Cell
Correctness	Primitive contracts: descriptors, data types, numerics	3/3
Selection	Choosing a primitive or the graph API	**2/3**
Integration	Framework layout boundaries	2/2
Debugging	Backend and numerical-parity triage	**3/4**
Performance	benchdnn, and the cost of reorders	2/2

Cell = implemented / total tasks that test this class.

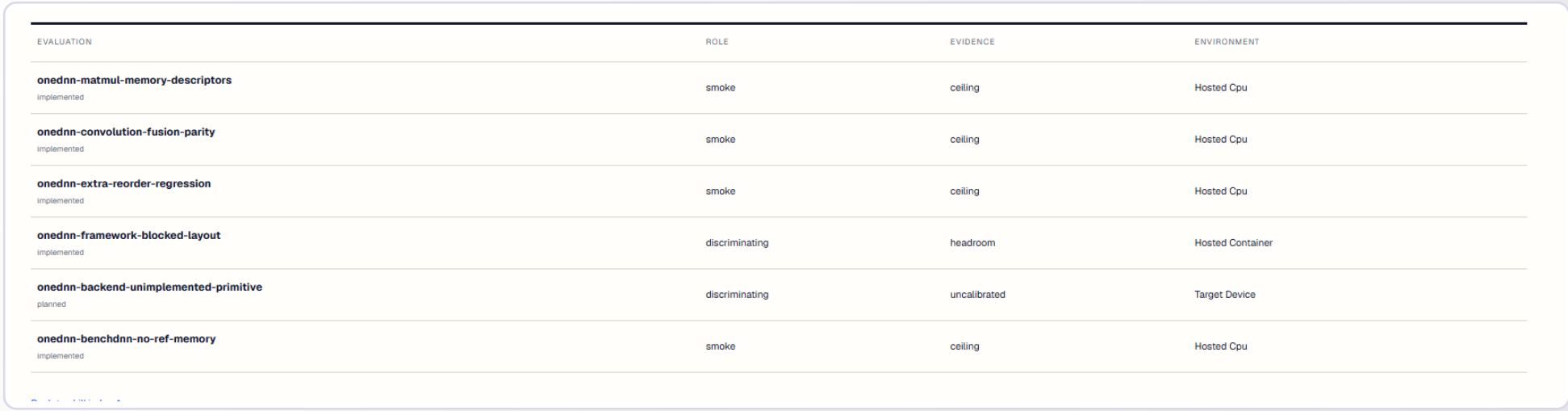
Green: all implemented. **Blue:** some still planned.

Columns add up to more than 6 because one task can test several classes.

Both blue cells come from the one planned task, backend-unimplemented-primitive.

Dashboard 3: the task list on each skill page

One row per evaluation task. Role is what the task was built for; Evidence is what the runs showed.



EVALUATION	ROLE	EVIDENCE	ENVIRONMENT
onednn-matmul-memory-descriptors implemented	smoke	ceiling	Hosted Cpu
onednn-convolution-fusion-parity implemented	smoke	ceiling	Hosted Cpu
onednn-extra-reorder-regression implemented	smoke	ceiling	Hosted Cpu
onednn-framework-blocked-layout implemented	discriminating	headroom	Hosted Container
onednn-backend-unimplemented-primitive planned	discriminating	uncalibrated	Target Device
onednn-benchdnn-no-ref-memory implemented	smoke	ceiling	Hosted Cpu

Evaluation

Task name, and whether it is implemented or planned.

Role

discriminating: built to show the skill's lift

smoke: a regression check

hardware: a device check

Evidence

headroom: skill raises the score

ceiling: all arms pass

no lift · uncalibrated (not run yet) · manual

Environment

Hosted Container: answer graded, no build

Hosted CPU: builds and runs real oneDNN

Target Device: needs specific hardware

Where evaluations run, and what triggers them

Workflow	Trigger	Runner	What it checks
validate	every PR, push to main	GitHub-hosted (ubuntu)	Catalog, card, evals format, links; oracle run of 23 tasks
harbor-hosted-toolchain	PR/push on its task, weekly (Tue ~2 AM ET), manual	GitHub-hosted	Oracle on the SYCL + oneDNN threading task (oneDNN #2959)
harbor-hardware	manual only	self-hosted Intel GPU	Oracle on SYCL device discovery
deploy-dashboard-pages	push to main on skills, tasks, results	GitHub-hosted	Rebuilds and publishes the dashboard
Model runs (with / without skill)	run by hand	local machine	compare_harbor_skill.ps1 → result report

"Oracle" runs the reference solution, so CI proves each task is solvable. It does not test agents. No model runs in CI, and no oneDNN task runs on a GPU yet.

Skill edits are not model-tested yet: who funds the runs?

A PR that edits a skill passes CI without any model ever running. A regression in agent results would go unnoticed.

On a skill PR today

- **validate:** format, links, oracle. The oracle ignores the skill.
- **dashboard:** rebuilds; publishes after merge.
- **Model runs:** only if someone runs the comparison by hand, on their own Codex account.

A

Repo secret + label
A maintainer adds a label; a workflow runs the edited skill's tasks with an API key. Forked PRs cannot see the secret.

B

Runner-control repo
The access-controlled repo that already gates GPU jobs holds the credential. Maintainers trigger it with a commit hash.

C

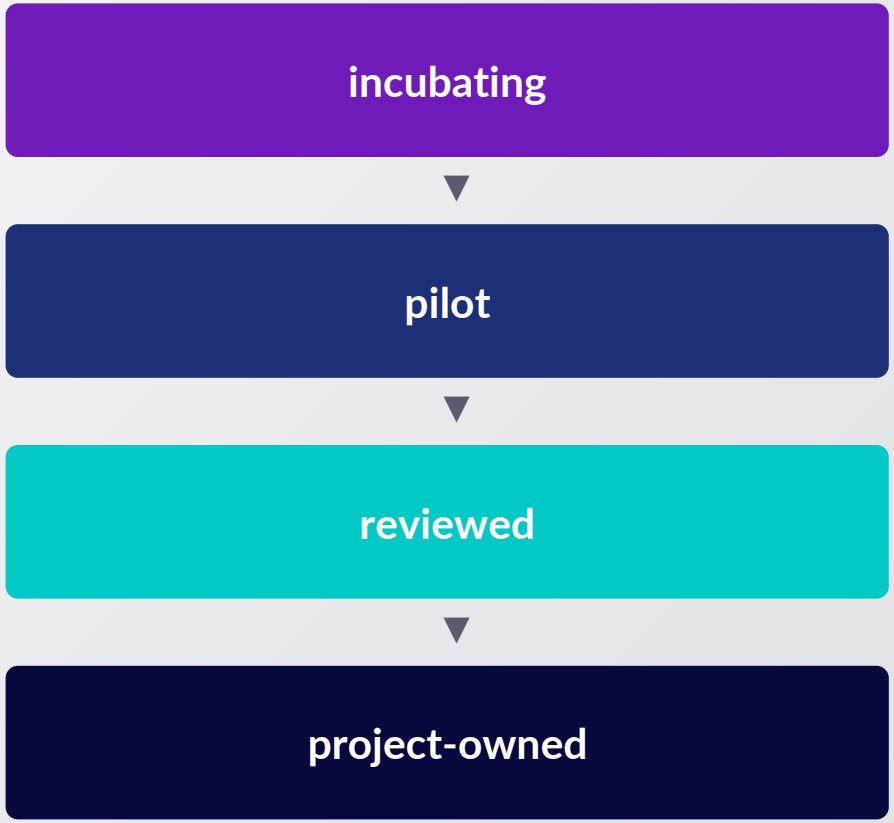
Scheduled run on main
Nightly or weekly. Catches regressions after merge, not before.

Open questions: whose account pays, which agents and models, and what budget per run. A full oneDNN promotion run is 5 tasks × 3 arms × 5 attempts = 75 trials.

What a oneDNN maintainer needs to know

The repo is incubating. Nothing here speaks for oneDNN until oneDNN reviews it.

- **Ownership:** CODEOWNERS routes skills/uxl-onednn/ to @uxlfoundation/onednn-maintainers.
- **Review:** docs/maintainer-review/uxl-onednn.md asks for a 30–45 minute accuracy review and a decision on ownership. State today: not requested.
- **Changing the skill:** open a PR. Update the skill, sources, evals, skills.yaml and card together.
- **Versions:** the skill does not pin a oneDNN release. It records a recheck date and tells agents to verify current upstream.
- **End state:** at project-owned, oneDNN's repo becomes the source of truth and this catalog mirrors it.



Promotion needs maintainer review and 5-attempt runs.

Two cross-cutting skills

Same shape as the library skills, but they serve every UXL project. Each adds a helper script.

BUILD AND RUNTIME

uxl-sycl-build-debug

Finds which phase failed: configure, compile, link, runtime load, or device selection. Then changes one thing at a time.

- sycl_probe.py: collects a toolchain and device report
- Gotchas: icpx, open DPC++ and AdaptiveCpp differ; a clean compile does not prove the device loads; a broad selector can silently pick the CPU; threading runtimes must match (oneDNN #2959)
- 8 tasks: 1 headroom, 3 ceiling, 1 no lift, 2 manual on Intel GPU, 1 uncalibrated

EVIDENCE AND CLAIMS

uxl-performance-validation

Turns “this should be faster” into evidence: correctness first, then fair timing, then a narrow claim.

- benchmark-contract.md: baseline, warmup, sync of async work, repeated runs, variance
- summarize_runs.py: turns benchmark CSV rows into per-case stats
- Hooks per library, e.g. benchdnn for oneDNN, data conversion for oneDAL, cgroup limits for oneTBB
- 4 tasks: 0 headroom, 3 ceiling, 1 no lift; 2 GPU tasks planned

Review for these goes through this group: no single project owns them.

Roundtable: questions for each project

1

Is the skill accurate?

What does it get wrong, or leave out, for your library?

2

What do agents get wrong?

Name one real issue or PR that would make a good evaluation task.

3

Where should it live?

In this catalog for now, or in your project repo once reviewed?

4

Who would review?

Would anyone like to do a 30–45 minute accuracy review?

Briefing deck per project: uxlfoundation.github.io/skills/decks/ (oneDNN, oneDAL, oneMath, oneTBB, oneDPL, oneCCL)

oneAPI plugins and a SYCL kernel skill

Biagio Cosenza



oneAPI plugins under UXL

Codeplay no longer ships prebuilt NVIDIA and AMD plugins. Users still need them.

What Biagio's group has done

- Users such as CINECA asked where to get the plugins they used to download from Codeplay.
- Worked out how to build them, with help from Intel and Argonne engineers.
- **Page:** unisa-hpc.github.io/oneapi-plugins, kept up to date by UNISA.
- Offered to move the page to UXL and maintain it.

Questions for the group

Home	Move the page into the uxlfoundation GitHub org?
Maintainer	Biagio's group as maintainer. What does the role commit to?
Scope	Build instructions only, or also prebuilt binaries and CI?
Steering	Does this need a Steering Committee decision?

A skill for writing SYCL kernels

The repo has a skill per library and one for SYCL build and debug. None teaches kernel writing.

The gap

- uxl-sycl-build-debug covers compile, link, runtime and device discovery
- Nothing covers writing a correct, fast kernel: work-group sizing, memory, sub-groups, USM vs. buffers
- Biagio offered to contribute; the Language SIG is a natural owner

How to add one

1. skills/uxl-sycl-kernels/: SKILL.md, references/, evals/
2. skill-cards/uxl-sycl-kernels.md and a skills.yaml entry
3. At least one Harbor task an agent fails without the skill
4. PR to uxlfoundation/skills; validators run on the PR

Hardware tasks can run on the qualified Intel GPU lane.

Open floor

Project updates and ideas for adapting to agentic AI

oneDNN

oneDAL

oneMath

oneTBB

oneDPL

oneCCL

One minute each. Anything that needs longer goes to the mailing list.

oneAPI DevSummit hosted by UXL Foundation 2026

December 1, 2026 · virtual · theme: the workload mix for agentic AI on heterogeneous compute

Oct 31

call for presentations closes this Friday

sessionize.com/oneapi-devsummit-hosted-uxl-foundation-2

What content would help most?

- **Each project:** what agents get right and wrong with your library today.
- **Skills:** a maintainer-reviewed skill and its evaluation results.
- **Plugins:** building and using oneAPI on NVIDIA and AMD GPUs (Language SIG).
- **Formats:** tutorials, demos, technical talks, panels.

An invitation: if your project has a story to tell, submissions are open until Friday.

Thank you

lists.uxlfoundation.org/g/open-source-wg

github.com/uxlfoundation/skills

uxlfoundation.github.io/skills

UXL

Unified Acceleration Foundation